Submission Worksheet

Submission Data

Course: IT202-007-F2025
Assignment: IT202 Milestone 1
Student: Roshan T. (rt524)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 11/10/2025 11:00:33 PM
Updated: 11/10/2025 11:00:33 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-1/grading/rt524
View Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-1/view/rt524

Instructions

- Overview Link: https://youtu.be/IDtqdaLwcVg

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `dev`
    5. On Github create a pull request from `dev` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout dev`
11. `git pull origin dev`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

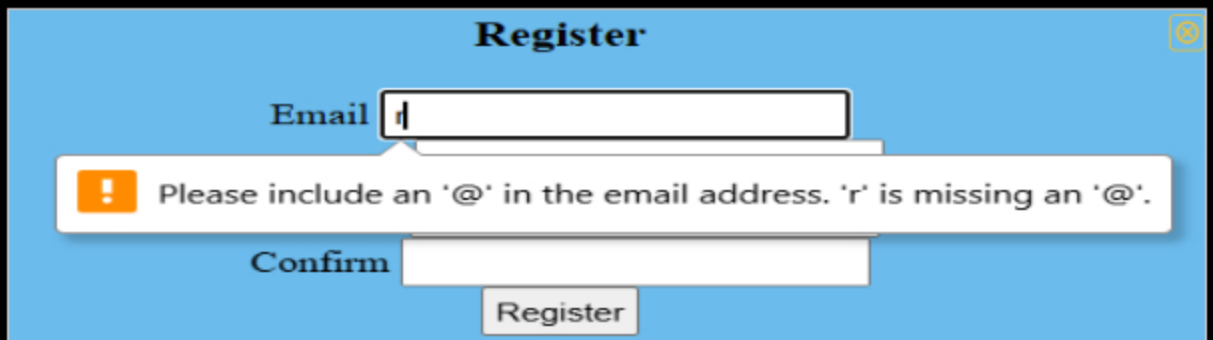
Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```php
<?php
require(__DIR__ . "/../../partials/nav.php");
?>
<h3>Register</h3>
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email</label>
        <input id="email" type="email" name="email" required />
    </div>
    <div>
        <label for="username">Username</label>
        <input type="text" name="username" required maxlength="60" />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <div>
        <label for="confirm">Confirm</label>
        <input type="password" name="confirm" required minlength="8" />
    </div>
    <input type="submit" value="Register" />
</form>
```

HTML validations register



email validation

password validation

Saved: 11/10/2025 9:21:57 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation here is the length of the passwords that must match the minimum requirement, which is 8 characters. This is done through minlength="8" in the label for the corresponding field.

Saved: 11/10/2025 9:21:57 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

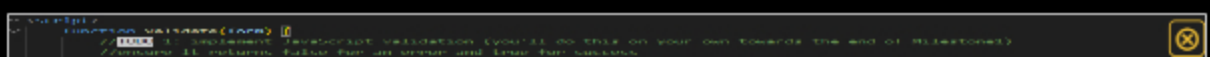
Progress: 100%

Part 1:

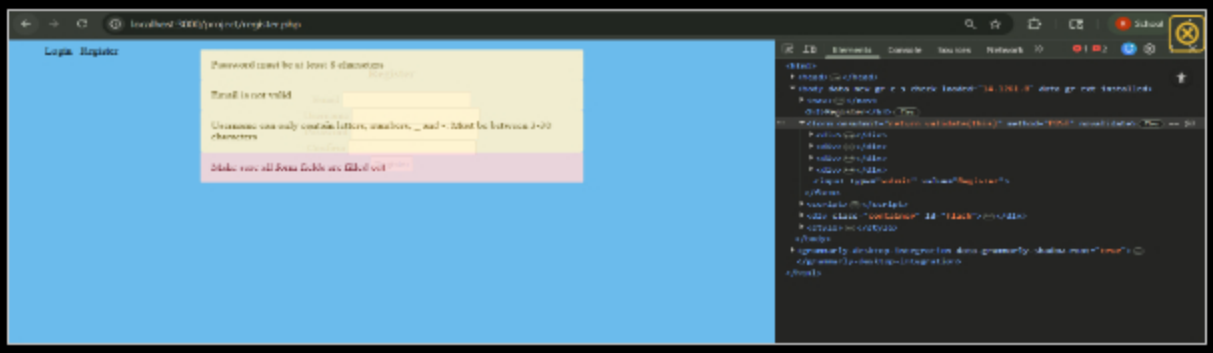
Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



javascript validate() function



js validation messages

Saved: 11/10/2025 10:56:25 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The function calls check the length being 8 characters as well as the validity of the email format, which is checked through regex. The username is also checked through regex and the message is displayed.

Saved: 11/10/2025 10:56:25 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

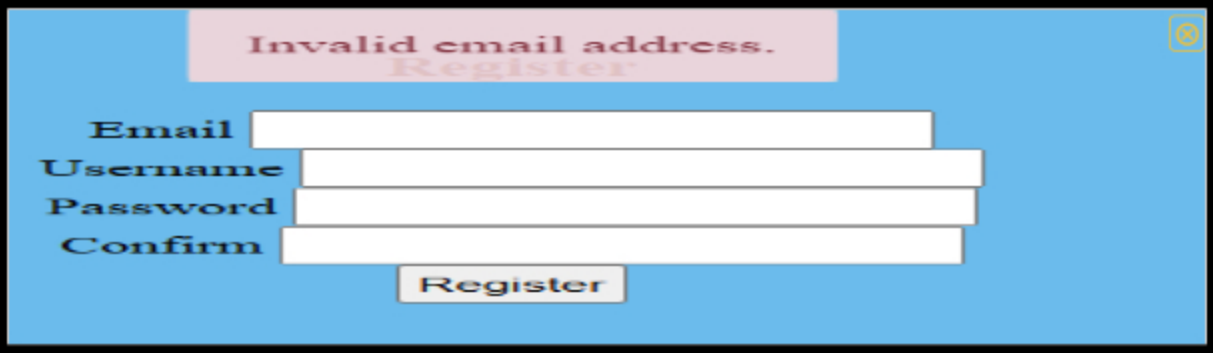
Progress: 100%

Details:

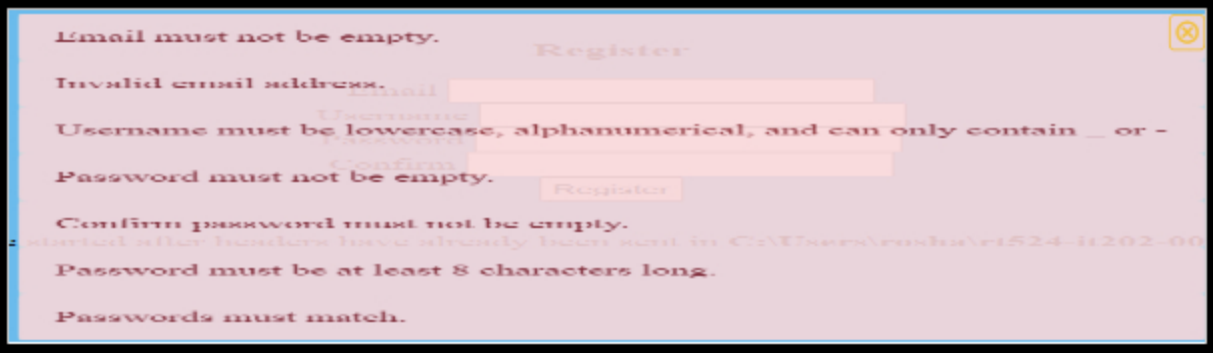
- Show the code related to the register page's PHP validation

- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

php validation



php invalid email



php validations for register

Saved: 11/10/2025 9:30:31 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the

requirement)

Your Response:

The fields of the form are checked if they are empty using empty(string) and if this returns true, provides a message. The email is first sanitized to ensure no injections can occur and then checked if it is valid using the is_valid_email function, the same being done to the password.

Saved: 11/10/2025 9:30:31 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/rt-524/rt524-it202-007/Milestone1/public_html/project/register.php
URL
https://github.com/rt-524/rt5

URL #2
https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/register.php
URL
https://rt524-it202-007-dev-5

URL #3
https://github.com/rt-524/rt524-it202-007/710
URL
https://github.com/rt-524/rt5

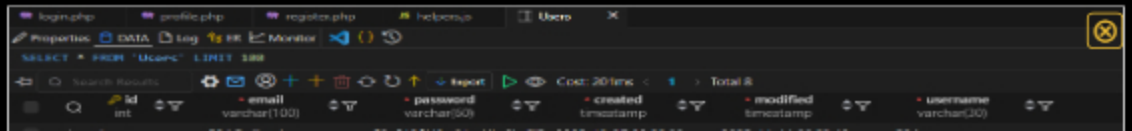
Saved: 11/10/2025 9:42:03 PM

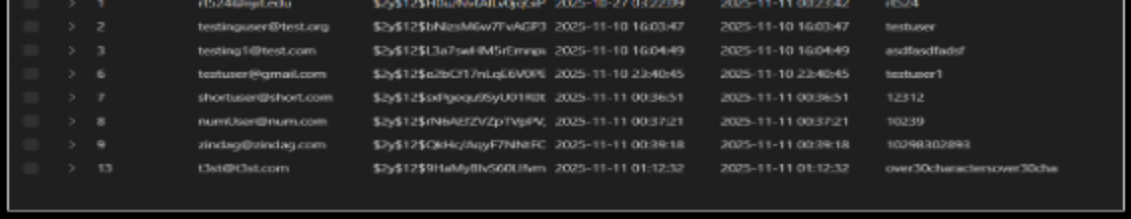
Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferrably a recent one during evidence capturing)





vscode database registered users

Saved: 11/10/2025 9:42:46 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

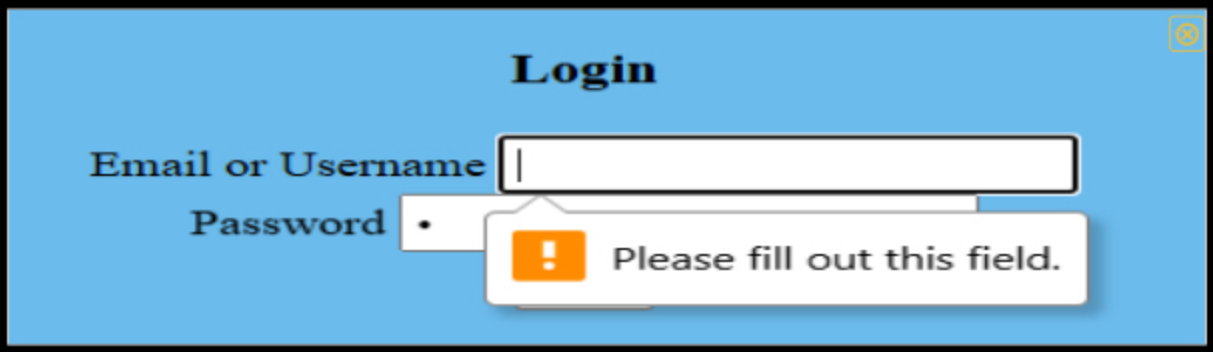
Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

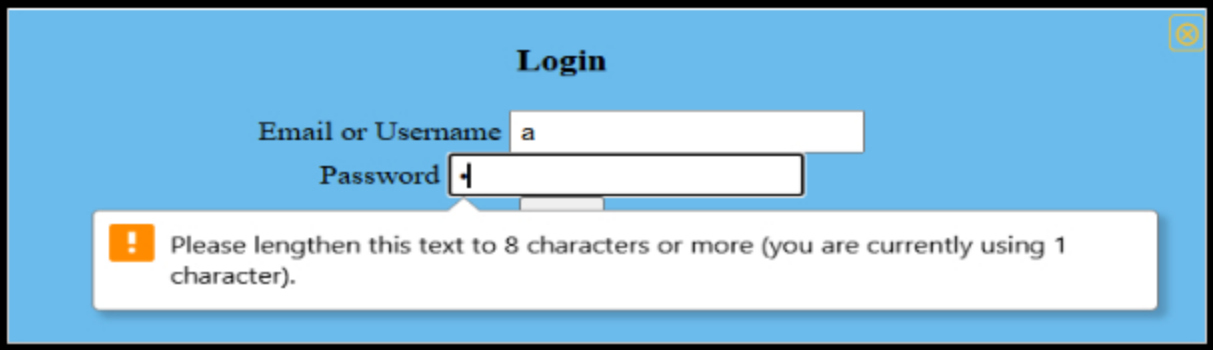
```php
<?php
require(__DIR__ . "/../../partials/nav.php");
?>
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST">
    <div>
```

```
        <label for="email">Email or Username</label>
        <input id="email" type="text" name="email" required />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <input type="submit" value="Login" />
</form>
```

html login form



html empty username



html short password

Saved: 11/10/2025 9:47:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

required length is 8 characters and the password is not, returns the length of the input and the required input

Saved: 11/10/2025 9:47:54 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

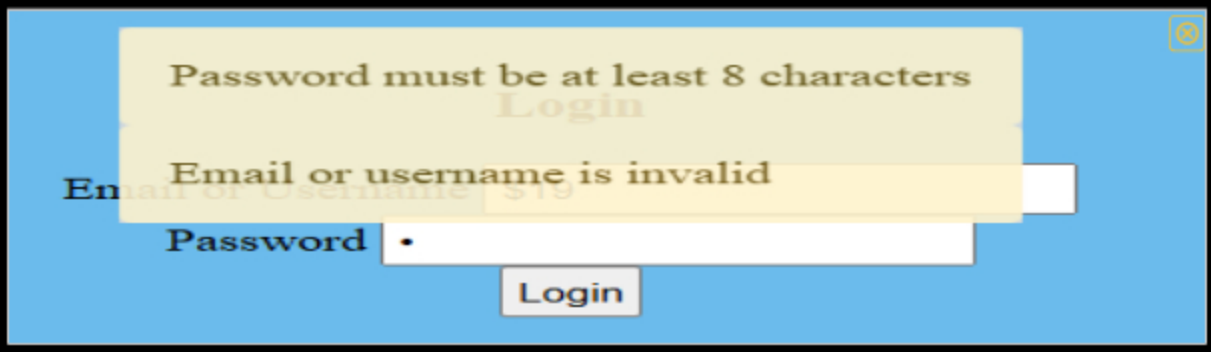
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



js validation messages

```
<script>
    function validate(form) {
        //TODO 1: implement JavaScript validation (you'll do this on your own towards the end of Milestone1)
        //ensure it returns false for an error and true for success
        // rt524 11/10 checks both isValidEmail and isValidUsername in helpra.js
        // uses regex to see if the input is valid for each field with valid characters
        // if neither is valid, will say one or the other is invalid, checks password length
        let login    = form.email.value;
        let password = form.password.value;
        // let isEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(login);
        // let isUsername = /^[a-zA-Z0-9_]{3,30}$/.test(login);

        if (!isValidPassword(password)) {
            flash("Password must be at least X characters", "warning");
            isValid = false;
        }
        if (!isValidEmail(login) && !isValidUsername(login)) {
            flash("Email or username is invalid", "warning");
            isValid = false;
        }
        return isValid;
    }
</script>
```

js validation code snippet

Saved: 11/10/2025 11:00:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

uses isValidEmail() && isValidUsername() with the same input to check if it is not one or the other using regex comparisons, then returns the error message. uses the same password checking as register with isValidPassword to check length.

Saved: 11/10/2025 11:00:33 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

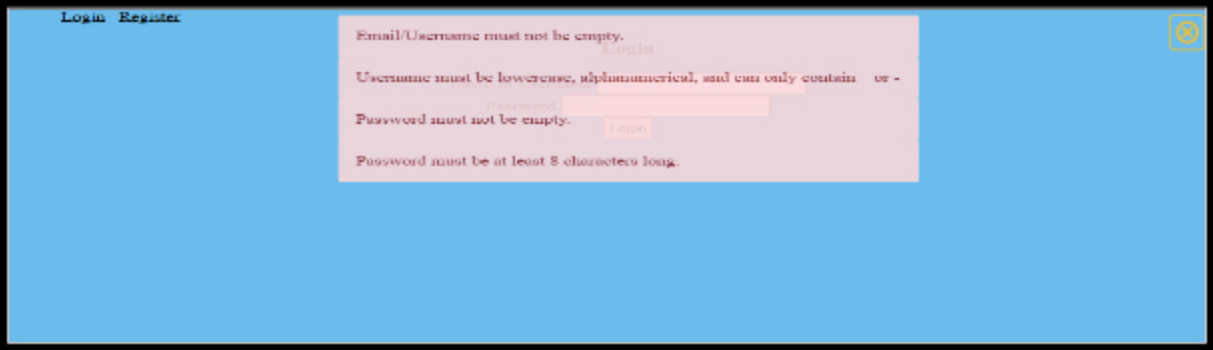
Part 1:

Progress: 100%

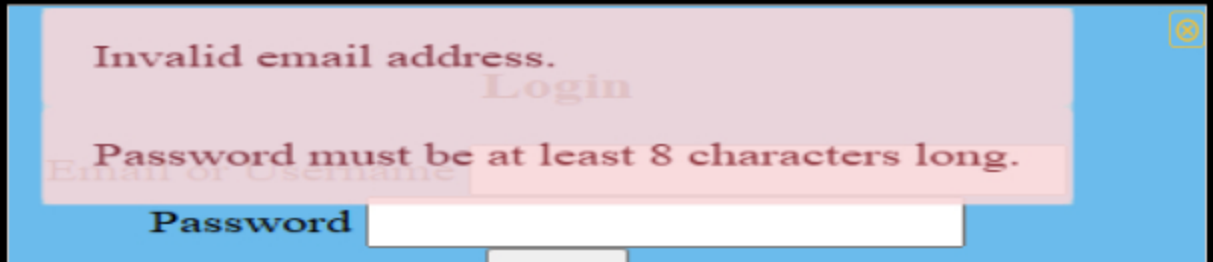
Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch

php validations login



php error messages



Login

invalid email address php validation

Saved: 11/10/2025 9:58:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

first checks if all fields are empty and provides error messages based on that. Then checks the sanitized email to ensure that it is in a valid format and prevent any possible injections. Otherwise can be treated as a username and checked to see if it follows the valid username format, and checks the validity of the password's length.

Saved: 11/10/2025 9:58:39 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/rt-524/rt524-it202-007/Milestone1/public_html/project/login.php
URL
https://github.com/rt-524/rt5

URL #2
https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/login.php
URL
https://rt524-it202-007-dev-5

URL #3
https://github.com/rt-524/rt524-it202-007/1
URL
https://github.com/rt-524/rt5

Saved: 11/10/2025 10:00:00 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

```
- From the heroku logs (Dashboard -> More button -> view Logs), capt
- It should show the id, username, email, and roles
```

heroku error log output

Saved: 11/10/2025 10:15:34 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

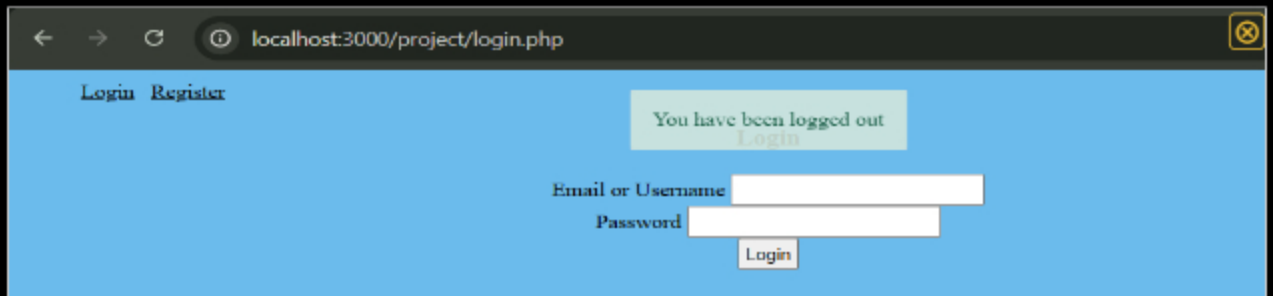
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message
- Show the code snippet of the logout page



logout message

public_html > project > logout.php
You, 5 hours ago | 1 author (You)

```php
<?php
// require functions.php to pull in flash()
require(__DIR__ . "/../../lib/functions.php");
reset_session(); // clear session data and start a new session
flash("You have been logged out","success");
header("Location: $BASE_PATH/login.php"); // redirect back to login
```

logout code snippet

Saved: 11/10/2025 10:06:03 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/rt-524/rt524-it202-001

URL
https://github.com/rt-524/rt524-it

Saved: 11/10/2025 10:06:03 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The session is first cleared entirely using reset_session() and a flash message is called to notify the user that they have been logged out. The flash function was included through the require function. After pressing the logout button, they are then redirected to the login page using BASE_PATH

Saved: 11/10/2025 10:06:03 PM

Section #4: (2 pts.) Feature: Security Rules

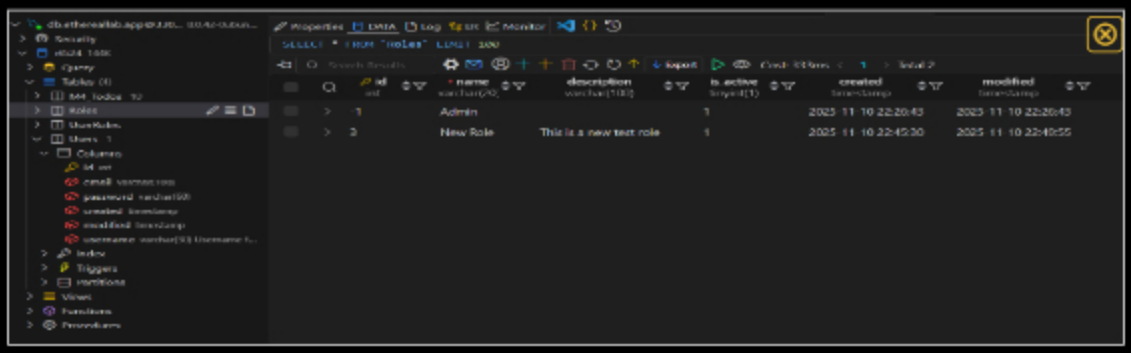
Progress: 100%

Task #1 (1 pt.) - Database Tables

Progress: 100%

Details:

```
- From the vs code mysql tool, show both the Roles and UserRoles tal
```



Roles table

user roles table

Saved: 11/10/2025 10:06:57 PM

Task #2 (1 pt.) - Demo Security Rules

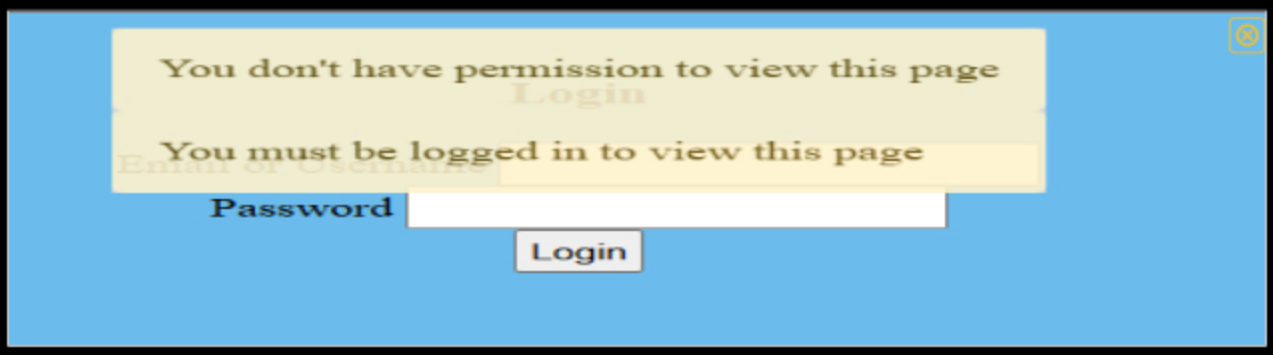
Progress: 100%

Part 1:

Progress: 100%

Details:

```
- Show the message related to needing to be logged in (i.e., try to manuall
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function
```



messages for needing to be logged in and proper perms

```php
function is_logged_in($redirect = false, $destination = "login.php")
{
    $isLoggedIn = isset($_SESSION["user"]);
    if ($redirect && !$isLoggedIn) {
        //if this triggers, the calling script won't receive a reply since die()/exit() terminates it
        flash("You must be logged in to view this page", "warning");
        $path = get_url($destination);

        die(header("Location: $path"));
    }
    return $isLoggedIn;
}
```

login check function

```php
if (!has_role("Admin")) {
    flash("You don't have permission to view this page", "warning");
    die(header("Location: " . get_url("landing.php")));
}
```

role check admin function

```php
function has_role($role)
{
    if (is_logged_in() && isset($_SESSION["user"]["roles"])) {
        foreach ($_SESSION["user"]["roles"] as $r) {
            if ($r["name"] === $role) {
                return true;
            }
        }
    }
    return false;
}
```

role check function

Saved: 11/10/2025 10:19:32 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/rt-524/rt524-it202p0718

URL
https://github.com/rt-524/rt524-it

Saved: 11/10/2025 10:19:32 PM

Part 3:

Progress: 100%

Details:

```
- Briefly explain how the login check code works
- Briefly explain how the role check code works
```

Your Response:

the login check first checks to see if you are logged in by seeing if the session for a user is set or not. If the session is not set, returns an error message and redirects you back. Otherwise, allows you to continue.

The role check code works by first checking if the user is logged in and if the role is set for a user. It then iterates through the users roles to get their names, otherwise it returns false if not applicable.

Saved: 11/10/2025 10:19:32 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

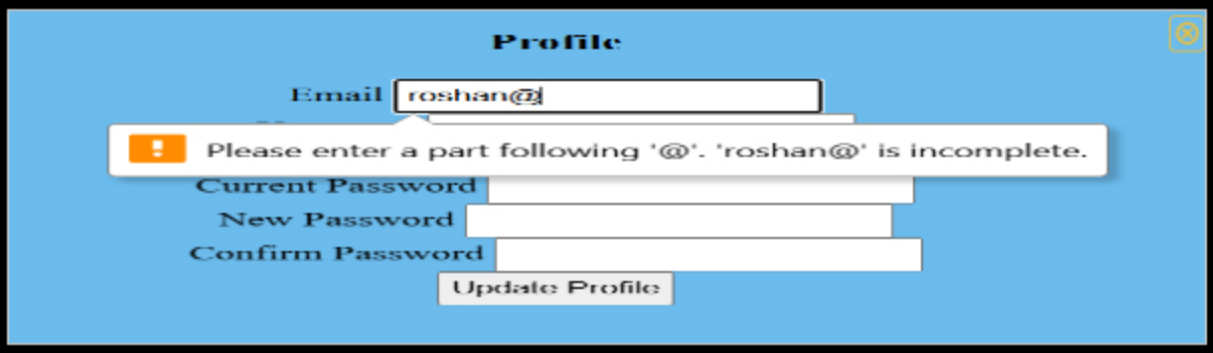
Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<h1>Profile</h1>
<form method="POST" onsubmit="return validate(this);">
    <div class="mb-3">
        <label for="email">Email</label>
        <input type="email" name="email" id="email" value="<?php echo($email); ?>" />
    </div>
    <div class="mb-3">
        <label for="username">Username</label>
        <input type="text" name="username" id="username" value="<?php echo($username); ?>" />
    </div>
    <!-- DO NOT PRELOAD PASSWORD -->
    <div>Password Reset</div>
    <div class="mb-3">
        <label for="cp">Current Password</label>
        <input type="password" name="currentPassword" id="cp" />
    </div>
    <div class="mb-3">
        <label for="np">New Password</label>
        <input type="password" name="newPassword" id="np" />
    </div>
    <div class="mb-3">
        <label for="conp">Confirm Password</label>
        <input type="password" name="confirmPassword" id="conp" />
    </div>
    <input type="submit" value="Update Profile" name="save" />
</form>
```

html code for profile



email @ error message

Saved: 11/10/2025 10:24:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

the validation simply checks if the email is valid by including text before and after the @ symbol.

Saved: 11/10/2025 10:24:33 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

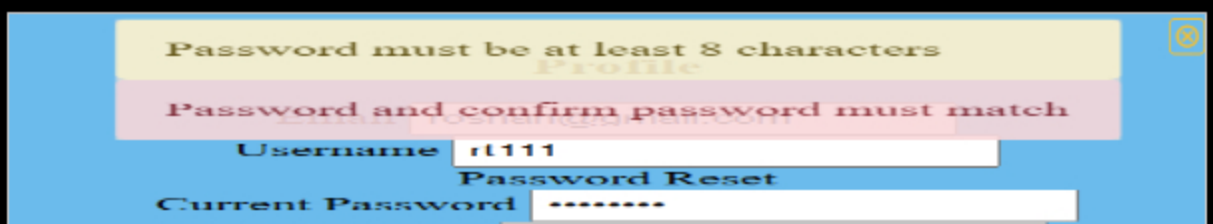
Progress: 100%

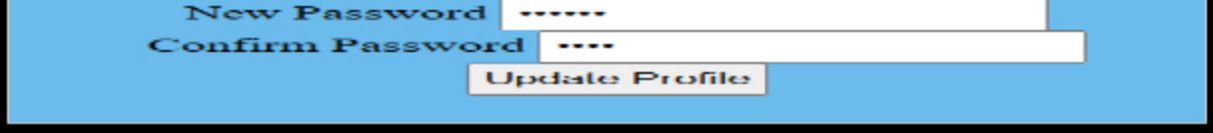
Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

code snippet of js validation on profile page

general error messages on empty





passwords dont match

Saved: 11/10/2025 10:58:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code first checks the length of the password with the !isValidPassword(pw) function. Then, individually checks the validity of the email and the username with their respective JS functions, checked with regex for validity.

Saved: 11/10/2025 10:58:27 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

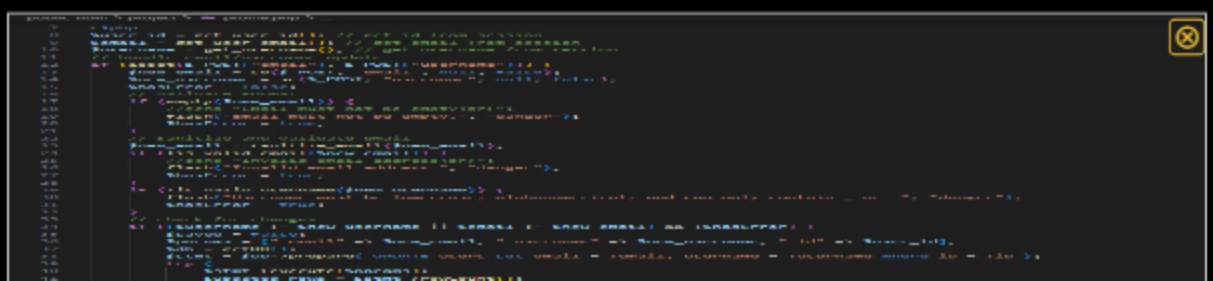
Progress: 100%

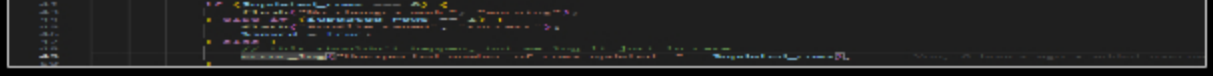
Part 1:

Progress: 100%

Details:

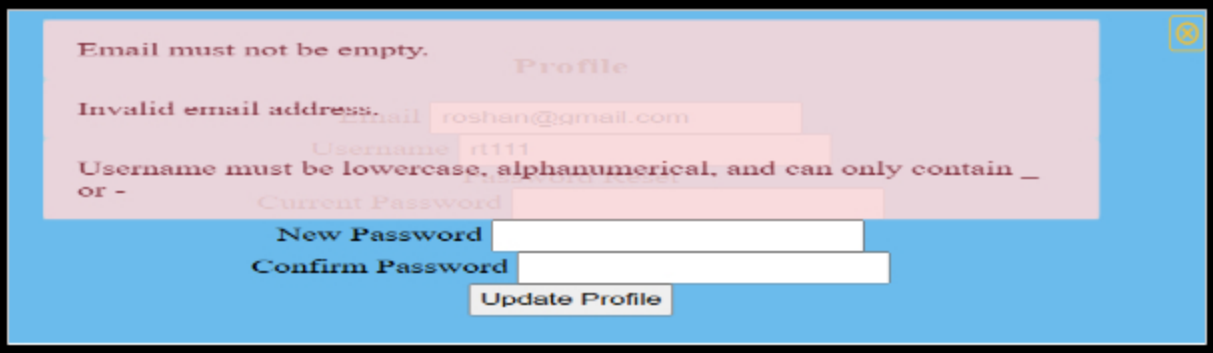
- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions



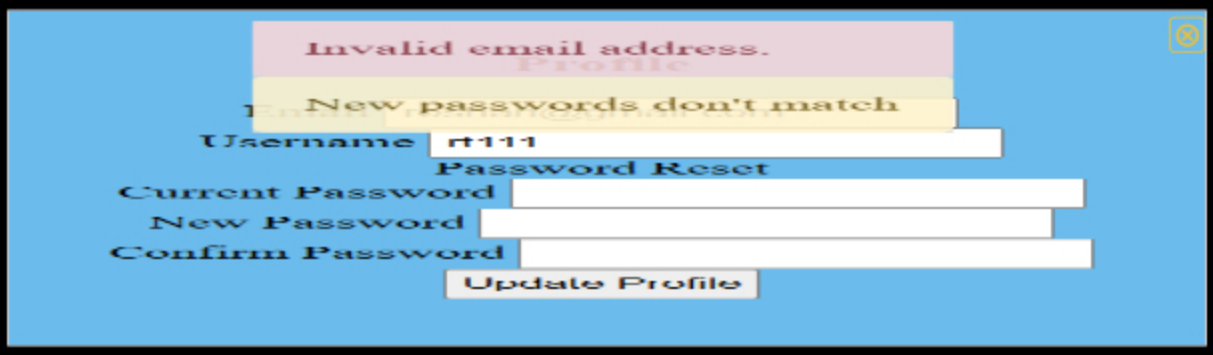


php validations part 1

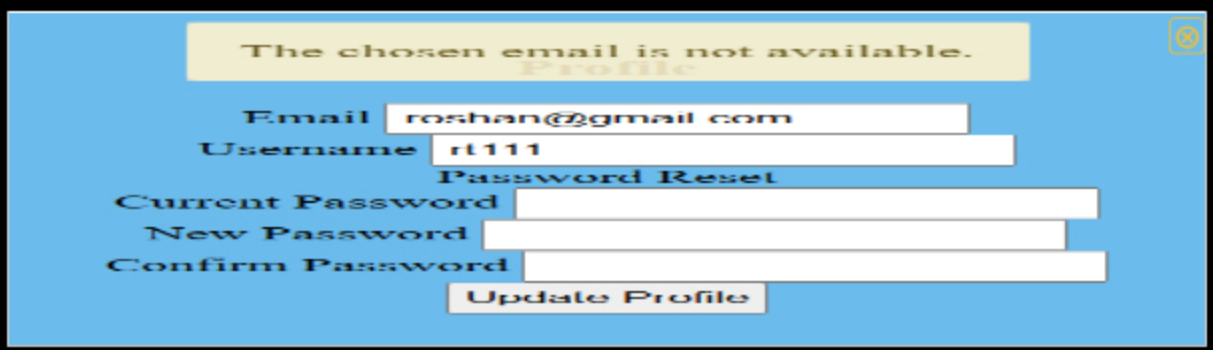
php validations part 2



php error messages part 1



php error messages part 2



php error messages part 3

php error messages part 4

Saved: 11/10/2025 10:40:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

from the session, first retrieve the user ID, email, and username. First check if the email and username are set before a potential change in username or email. Do the standard validations on the new email entered as well as the username, checking for validity, emptiness, and uniqueness. Also checks for matching password to confirm any password updates and checks validity of those new passwords and matches against the confirmation.

Saved: 11/10/2025 10:40:27 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

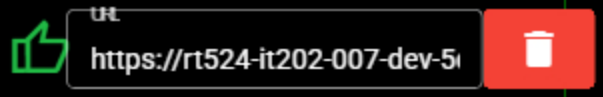
URL #1

https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/%3C?php%20require_once(__DIR__%20.%20
%22,%20%22danger%22);%20%20%20%20%20%20%20%20%20$hasError%20=%20true;%20%20%20%20%2
%3ErowCount();%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20if%20($updated_rows%20===%200)
%3Efetch(PDO::FETCH_ASSOC);%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20if%2
[%22username%22]%20=%20$user[%22username%22];%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%
%3Eprepare(%22SELECT%20password%20from%20Users%20where%20id%20=%20:id%22);%20%20%20%20%2
%3Eexecute([%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%

%3ErowCount();%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%2
%3E%20%3Ch3%3EProfile%3C/h3%3E%20%3Cform%20method=%22POST%22%20onsubmit=%22return%2
div%3E%20%20%20%20%20%3Cdiv%20class=%22mb-3%22%3E%20%20%20%20%20%20%20%20%20%3Cl
%20Must%20be%20between%203-30%20ch[illegible]%20%20%20%20%20%20%20%20%20isValid%

URL #2
https://github.com/rt-524/rt524-it202-007/tree/Milestone1/public_html/project/profile.php

URL #3
https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/login.php



URL #4
https://github.com/rt-524/rt524-it202-007/tree/Milestone1/public_html/project/profile.php

Saved: 11/10/2025 10:42:28 PM

Section #6: (1 pt.) Misc

Progress: 100%

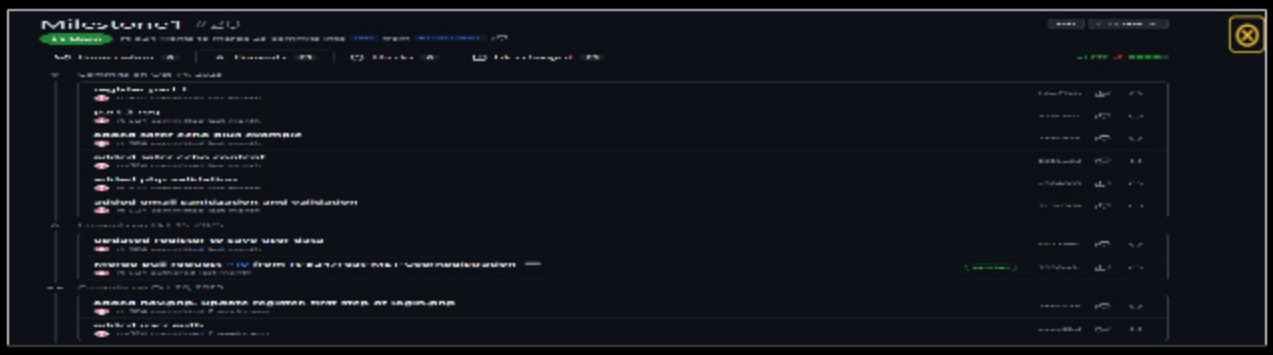
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



commits part 1

commits part 2

Saved: 11/10/2025 10:46:32 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/rt-524/rt524-it202-007

URL
https://github.com/rt-524/rt524-it

Saved: 11/10/2025 10:46:32 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

wakatime overall

wakatime individual

Saved: 11/10/2025 10:48:12 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Throughout this milestone, I learned the necessity of validation in every part of a form. This can be through HTML, JavaScript, and PHP which all provide different layers of safety. HTML and JavaScript ensure that the initial values of the form submit with the proper values. The PHP side of validation checks the validity in terms of the database, first sanitizinge everything to prevent SQL injections and then providing an additional layer of validation before putting anything into the database that the client side validations may have missed.

Saved: 11/10/2025 10:50:30 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easieset part of this assignment was going through the SQL queries and dealing with the databases as I am most familiar with this as I'm taking CS331 at the same time. This allowed me to further understand how the database works on a deeper level as well as how most sides would go through their validations and database setup in terms of registration, logins, and updates for profiles.

Saved: 11/10/2025 10:51:30 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment for me was keeping track of all the different functions and when I should be using repeats of things, making functions of repeated things, or how certain functions are accessed. Sometimes, I did not fully understand why a new file or function needed to be made, but once we started to add more functionality and those functions or files were used more, I saw that they were necessarily when scaling our project more.

Saved: 11/10/2025 10:52:43 PM